飞机结构FEA数据集 · 典型用例与使用流程指南

面向「LLM + Agent 的飞机结构强度分析有限元模型高效构建与更新」研究

配套《飞机结构FEA数据集调研报告》· 基于本地已下载数据集的真实文件结构编写 · 2026-07-14

> **本指南目的**：不说空话。每个数据集都给出「是什么 → 数据真实长什么样 → 典型用例 → 可落地的步骤化流程（含命令/代码）→ 与飞机结构FEA课题的适配性」。阅读完即可照着在本地复现。

0. 数据集一览与定位对比

下方四个基准构成了你课题在「模型构建」与「模型解读」两端的评测/训练素材。**ALL-FEM、FEM-Bench、StructuralEval 三个已下载到本地**（见 `datasets/`），**FRAME 暂无公开下载链接**，本节按真实情况标注。

数据集	核心任务定位	数据形态（真实）	规模	许可	本地状态	综合适配
ALL-FEM	代码生成 + 自主仿真（FEniCS 求解脚本）	`alpaca_format.json`：1005 条 instruction/input/output	1.8 MB	论文开源	已下载	3.0 / 5
FEM-Bench	代码生成数值正确性评测（FEM 函数级）	`tasks/` 33 题 + `prompts/` + 参考解 + `run_pipeline.py`	87 MB	见 LICENSE	已下载	3.0 / 5
StructuralEval	图片→结构模型 JSON 评测（梁/刚架/桁架）	`data/raw_models` 25 题 + `ground_truth_meta` + `bin/framecalc.wasm`	74 MB	**CC BY-NC 4.0**	已下载	3.0 / 5
FRAME	FEA 结果解读 + 自优化建议（含航空航天）	论文声称开源，但无公开仓库/下载链接	未知	待确认	未下载	3.5 / 5

> **关键认知**：四个基准**全部聚焦「构建 / 解读 / 代码生成」，没有一项直接覆盖课题后半段「有限元模型更新（试验/监测数据驱动的参数反演与修正）」**。模型更新数据来自调研报告 §2（Hawk T1A 全机模态、M001 CRM 折叠翼 GVT + 更新后模型、复合材料导波库等）。本指南第 6 节给出把「构建栈」与「更新栈」串起来的编排思路。

1. ALL-FEM 使用指南

代码生成 · 监督微调 · 自主仿真智能体

1.1 它是什么

ALL-FEM 是面向**有限元求解代码生成**的指令微调数据集，底层基于 FEniCS（`dolfin`）PDE 求解框架。论文同时提出了一套**多智能体自主仿真框架**（问题理解 Agent、代码生成 Agent、验证 Agent 等），并在 39 道基准题上报告：GPT-OSS-120B 成功率约 **71.79%**。数据集本体就是一个 JSON 列表。

1.2 数据真实长什么样

文件 `datasets/ALL-FEM/alpaca_format.json`，长度 1005，每条含三个字段：

字段	含义	示例（真实截取）
`instruction`	高层任务描述（自然语言）	"Develop a Python script using the FEniCS library to solve the 2D Poisson equation on a rectangular domain with Dirichlet boundary conditions"
`input`	问题约束/参数补充	"The domain is a rectangle discretized into a mesh of 6 by 4 elements. The boundary conditions are defined by the expression u_0 = 1 + x[0]^2 + 2*x[1]^2. The source term f is a constant −6.0"
`output`	完整可运行 FEniCS 脚本（含 ```` ```python ```` 代码块）	"```python from dolfin import * mesh = UnitSquare(6, 4) V = FunctionSpace(mesh, 'Lagrange', 1) ...```"

覆盖题型示例：2D Poisson、Dirichlet/Neumann 边界、Picard/Newton 非线性迭代、Jacobian 显式定义、1D 问题等。

1.3 典型用例

用例 A：监督微调（SFT）让 LLM 生成 FEniCS 求解脚本

- 把 1005 条 `instruction+input → output` 当作 SFT 样本，微调 Qwen/Llama 等开源基座，使其学会「读题 → 写 FEniCS 求解器」。
- 产出：一个面向计算力学的「建模代码助手」，可作为你「建模 Agent」的代码生成子模块。

用例 B：构建多智能体自主仿真框架

- 复现论文范式：问题理解 Agent 拆解需求 → 代码生成 Agent 产出脚本 → 验证 Agent 运行并纠错（self-correction loop）。
- 产出：给定一段强度分析需求描述，自动产出并可执行的有限元求解脚本。

1.4 使用流程（用例 A：SFT）

1. **加载并清洗数据**：剥离 `output` 里的 Markdown 代码围栏，得到纯 Python 脚本用于执行验证。
2. **切分训练/验证集**：按 9:1 随机切分，保留验证集做过拟合监控。
3. **格式化为 SFT 对话**：拼成 `system + user(instruction+input) + assistant(output)` 。
4. **微调**：用 HuggingFace `transformers` / `trl` 做 LoRA 或全参微调。
5. **推理 + 执行校验**：对新题生成脚本，`subprocess` 运行，捕获异常做自愈重试。

```
# 1) 加载与清洗
import json, re
data = json.load(open("datasets/ALL-FEM/alpaca_format.json"))
def strip_fence(s):
    m = re.search(r"```python\s*(.*?)```", s, re.S)
    return m.group(1).strip() if m else s.strip()
samples = [{
    "instruction": d["instruction"], "input": d["input"],
    "code": strip_fence(d["output"])
} for d in data]
print(len(samples), "samples; e.g. ->", samples[0]["code"][:60])

# 2) 微调（LoRA 示例，伪代码）
from trl import SFTTrainer
from datasets import Dataset
ds = Dataset.from_list([{
    "messages": [
        {"role":"system","content":"You are an expert FEniCS FEM engineer."},
        {"role":"user","content":f"{s['instruction']}\n{s['input']}"},
        {"role":"assistant","content":s["code"]},
    ]} for s in samples])
trainer = SFTTrainer(model="Qwen/Qwen2.5-Coder-7B", train_dataset=ds,
                     peft_config=lora_cfg, max_seq_length=4096)
trainer.train()

# 3) 推理并执行校验
import subprocess, tempfile, os
code = generate(user_prompt)                # 模型输出
p = tempfile.NamedTemporaryFile("w", suffix=".py", delete=False); p.write(code); p.close()
r = subprocess.run(["python", p.name], capture_output=True, text=True, timeout=120)
print("OK" if r.returncode == 0 else f"ERR: {r.stderr[:300]}")
```

适配性提醒：ALL-FEM 基于 **FEniCS（开源、弱形式 PDE）**，与飞机强度分析主流的 **NASTRAN / Abaqus（BDF/INP、强形式、工业求解器）**存在明显鸿沟。直接迁移需做：① 把 FEniCS 弱形式教学题**映射到 aircraft 几何/材料/载荷**；② 或把「代码生成」目标换成 NASTRAN 卡片生成（可借助调研报告 §2 的现成 FE 模型作锚点）。但作为「**建模 Agent 的代码生成能力底座**」仍 highly valuable。

2. FEM-Bench 使用指南

代码生成评测 • 数值正确性 • A/B 研究

2.1 它是什么

FEM-Bench 是**计算力学代码生成基准**，评测 LLM 在给定「函数签名 + docstring」时能否写出**数值正确**的有限元函数，并用 pytest 对照参考解做数值比对（非字符串匹配）。共 **33 个任务**，分三类：

类别	代表任务	考察点
FEM1D	`linear_uniform_mesh_1D`，`element_stiffness_linear_elastic_1D`，`solve_linear_elastic_1D_self_contained`	一维网格、单元刚度、线弹性求解
FEM2D	`generate_quad8_rectangular_mesh`，`quad8_shape_functions_and_derivatives`，`quad_quadrature_2D`，`compute_physical_gradient_quad8`	单元网格、形函数、数值积分
MSA3D	`local_elastic_stiffness_matrix_3D_beam`，`solve_linear_elastic_frame_3D`，`eigenvalue_analysis_msa_3D`，`elastic_critical_load_analysis_frame_3D`	三维梁/刚架、特征值(屈曲)、几何刚度

2.2 数据真实长什么样

- `tasks/*.py`：每个任务一个文件，内含**带 docstring 的目标函数** + **pytest 测试**（参考解）。例 `linear_solve(P_global, K_global, fixed, free)` 返回位移与节点反力，含 `cond(K_ff) <1e16` 数值稳定性检查。
- `prompts/*_code_prompt.txt`：给模型的「函数实现任务」提示词，含签名、docstring、允许 import、禁止改 helper。
- `prompts/*_test_prompt.txt`：要求模型为该函数写 pytest 测试。
- `run_pipeline.py`：主流程（生成代码 → 生成测试 → 跑 pytest → 数值比对 → `accuracy_at_k` 汇总）。
- `llm_api/llm_clients.py`：`call_llm_for_code` / `call_llm_for_tests`，走 OpenAI 兼容接口。

```
# prompts/linear_solve_code_prompt.txt（真实格式）
# Python Function Implementation Task
# Write a Python function that matches the exact signature and docstring provided below.
## Requirements:
# - Keep the function name, parameter names, and docstring exactly as shown
# - Use only the following imports: import numpy as np ; import pytest
# - Output only valid Python code (no explanations, comments, or markdown)
## Function Signature:
def linear_solve(P_global, K_global, fixed, free): ...
```

2.3 典型用例

用例 A：评测你的建模/求解 Agent 的代码正确性

- 把待评测模型（或你的 Agent 代码生成模块）接入 `run_pipeline.py`，输出 33 题的 pass@k 与数值误差分布。
- 直接产出可写进论文的「能力基线表」。

用例 B：研究方法 A/B 对照

- 仓库自带 `SYSTEM_PROMPT_STUDY.md` / `TEMPERATURE_STUDY.md`：对比「有无系统提示」「不同温度」对正确率的影响。
- 可复用其框架做「带/不带 RAG（材料库/手册）」对照。

2.4 使用流程（跑通官方评测）

1. **装依赖**：`pip install -e .`（或按 `pyproject.toml` 安装 `fem_bench`、`openai`、`numpy`、`pytest`）。
2. **配置模型端点**：修改 `run_pipeline.py` 的 `MODEL_NAMES`，并让 `llm_api/llm_clients.py` 指向你的 OpenAI 兼容服务（含本地 vLLM）。
3. **选择实验组**：切 `with_system_prompt` 开关、调 `TEMPERATURE`、`SEED`。
4. **运行**：`python run_pipeline.py` → 代码/测试生成到 `llm_outputs_*/`。
5. **汇总**：`accuracy_at_k_table.py` 读取 `results_*/` 产出 pass@k 表。

```
# 极简自跑：不用官方 pipeline，直接对单题做数值比对
import numpy as np, importlib.util, pytest
task = "tasks/linear_solve.py"            # 含参考函数 + 测试
spec = importlib.util.spec_from_file_location("t", task)
mod = importlib.util.module_from_spec(spec); spec.loader.exec_module(mod)
# 让模型补全 mod.linear_solve 的函数体 -> predicted.py
pred = importlib.util.module_from_spec(importlib.util.spec_from_file_location("p","predicted.py"))
# 用 pytest 对比：断言 predicted 与参考解在 tol 内一致
np.testing.assert_allclose(pred.linear_solve(*args), mod.linear_solve(*args), rtol=1e-6)
```

适配性提醒：FEM-Bench 是**函数级、教科书式**难度（单元刚度、形函数、网格），不含「完整建模链路（几何→网格→属性→工况→求解→后处理）」，也**不含模型更新**。但它非常适合作为你「**建模 Agent 的代码正确率硬性门槛**」——先过 FEM-Bench，再进阶到 ALL-FEM 的端到端脚本与真实飞机 FE 模型。

3. StructuralEval 使用指南

多模态 • 图片→结构JSON • 物理引擎验证

3.1 它是什么

StructuralEval 评测**多模态大模型（LMM）**能否扮演「建模工程师」：输入一张结构图（梁/刚架/桁架），输出计算机可识别的标准结构定义 **JSON**，再由内置 **WASM 求解器 FrameCalc** 做有限元分析，比对其物理响应（支座反力、内力、弯矩）与真值，判定模型是否「真读懂了结构」。核心理念：**「AI 负责建模，引擎负责计算」**。难度分级 1–5（简支梁 → 复杂超静定刚架）。

3.2 数据真实长什么样

路径	内容
`data/raw_models/*.json`	25 题建模文件（`beam_*` / `frame_*` / `truss_*`）。Schema：`points[{id,x,y}]`、`links[{id,a,b,E,A,Iz,density,endA,endB}]`、`supports[{id,at,type}]`、`loads[...]`
`data/ground_truth_meta/*.json`	真值：`{id, difficulty(1-5), image_filename, solution:{reactions:[{atId,type,value}], axial:[{linkId,samples:[{s,n,v,m}]}], ...}}`
`data/images/*.png`	题目图片
`bin/framecalc.wasm`	WASM 物理求解器（FrameCalc）
`run_eval.py` + `src/` + `tools/`	评测主程序、加载器、求解桥、指标、Prompt 注册表

```
// data/raw_models/beam_001.json（真实节选）
{
  "points": [{"id":"P1","x":0,"y":0}, {"id":"P2","x":2,"y":0}, {"id":"P3","x":4,"y":0}],
  "links":  [{"id":"L1","a":"P1","b":"P2","E":80918000,"A":0.00785,"Iz":4.9e-6,"density":7850,"endA":"rigid","endB":"hinge"}],
  "supports":[{"id":"S1","at":"P1","type":"pin"}],
  "loads":[{"id":"F1","at":"P2","type":"fy","value":-10}]
}
// data/ground_truth_meta/beam_001.json -> solution.reactions[].value 即物理真值
```

3.3 典型用例

用例 A：评测多模态 LLM 的「读图建模」能力

用例 B：作为「建模 Agent」的零件

- 用 `run_eval.py` 接入任意 OpenAI 兼容 VL 模型，输出 `<json>...</json>`，自动喂 WASM 求解并比对，得到 physics-based 准确率。
- 示例已验模型：GPT-4o、Gemini、Qwen-VL、Claude。
- 把「图纸/截图 → 结构 JSON → FrameCalc 求解」封装成你 Agent 的一个工具节点，实现从视觉输入到可计算模型的自动化。
- FrameCalc 在线工具（`framecalc.aeutlook.com`）兼容该 JSON，便于可视化校核。

3.4 使用流程（评测 / 接入）

1. **装依赖**：`pip install -r requirements.txt`（含 `openai`，`tqdm`，`wasmtime`，`json_repair`）。
2. **自检（无需模型）**：`python run_eval.py --debug` 用真值 JSON 跑通整条 WASM 求解→打分链路，验证环境。
3. **正式评测**：指向 OpenAI 兼容端点（云端或本地 vLLM `http://localhost:8000/v1`）。
4. **过滤 / 并发**：`--filter beam` 只跑梁类，`--limit 5` 抽样，`--concurrency 4` 并行。
5. **读结果**：脚本输出各题 reactions/内力对齐分数，聚合成加权准确率。

```
# 本地 vLLM 部署一个 VL 模型后直接评测（真实命令格式）
python run_eval.py --model "qwen-vl-plus" \
  --api-base "https://dashscope.aliyuncs.com/compatible-mode/v1" \
  --api-key "sk-..." --max-retries 3
# 仅验证环境（不调模型）
python run_eval.py --debug
# 只测梁类、并发 4
python run_eval.py --model "gpt-4o" --api-key "sk-..." --filter "beam" --concurrency 4
```

适配性 + 许可提醒：① StructuralEval 仅覆盖**杆系（梁/刚架/桁架）**，不含板壳/实体/复合材料，与飞机机翼壁板、机身蒙皮的**连续体有限元**有差距；但「**图片/图纸→结构化模型 JSON**」这一范式对「建模 Agent 的视觉输入前端」极有参考价值。② 许可为 **CC BY-NC 4.0（非商用）**，仅限研究，商用需作者授权。

4. FRAME 使用指南（含获取限制说明）

FEA 结果解读 · 自优化建议 · 暂无公开下载

4.1 它是什么（基于调研报告定位）

FRAME（*Finite element Reasoning and AI-Agent Model Engine*）是面向 **FEA 结果解读**的基准，强调对有限元分析**后处理结果**（云图、数值、报告）的理解与推理，并给出**自优化建议。**论文明确**包含航空航天**场景，设三类任务：① **识别（Identify）** 异常/高应力区；② **提取（Extract）** 关键数值与结论；③ **建议（Recommend）** 改型/优化方向。数据规模达数千数据点。

4.2 预期数据形态与任务

任务	输入	期望输出	在你课题里的角色
Identify	FEA 结果（云图/数值表）	高应力/危险区域标注与定性判断	模型更新前的「问题定位」
Extract	FEA 报告片段	结构化关键量（最大 von Mises、位移、安全裕度）	更新 Agent 的「指标抽取」
Recommend	上述两者	改型/局部加厚/材料替换等优化建议	更新 Agent 的「建议生成」

4.3 典型用例

用例 A：FEA 结果智能解读 → 自优化建议

- 把 NASTRAN/Abaqus 后处理结果喂给模型，自动产出「哪里危险、差多少、怎么改」的可读报告。
- 直接服务你课题的「**模型更新**」闭环（解读 → 定位 → 建议修正参数）。

用例 B：作为「更新 Agent」的校验/建议模块

- 在「试验数据驱动参数反演」之后，用 FRAME 风格的解读能力评估更新后模型是否合理、给出进一步调优方向。

4.4 使用流程（预期；因无下载，先列待确认步骤）

1. **获取数据集**：在 OpenReview 论文页（`forum?id=2TdYc5V4jU`）或作者仓库找到下载链接（当前页面有浏览器验证、且本沙箱 github.com 不可达）。
2. **本地化读取**：按论文 supplementary 把 FEA 结果样本与标注载入。
3. **接入评测**：仿照三任务（Identify/Extract/Recommend）构造 prompt，调用模型并比对标注。
4. **嵌入更新闭环**：与 §2 报告的试验数据（Hawk T1A / M001 CRM）配合，形成「解读→建议→再分析」回路。

获取限制（重要）：经多轮检索，FRAME **没有公开可下载链接**——公开搜索仅命中同名 CVPR 虚拟人动作数据集；其 OpenReview 论文页与 PDF 均有浏览器验证拦截；且即便有 GitHub 链接，本沙箱 **github.com 不可达**（仅 hf-mirror.com / Zenodo / arXiv 可连通）。**当前未下载。**若你拿到其 HuggingFace / Zenodo / 网盘等可访问链接，我可立即补下。

5. 综合：面向「建模 + 更新」的智能体工作流编排

把四个数据集串成你的研究底座

课题分两段：**构建**（几何/CAD→网格→属性→求解脚本）与 **更新**（试验/监测数据→参数反演→修正模型）。四个基准主要服务「构建 + 解读」，更新数据在调研报告 §2。建议如下编排：

5.1 构建栈（用 ALL-FEM + FEM-Bench 训练/评测建模 Agent）

阶段	用到的数据集	动作
能力底座	ALL-FEM（SFT）	微调 LLM 生成 FEniCS 求解脚本 → 建模 Agent 的代码生成内核
硬性门槛	FEM-Bench（评测）	用 33 题 pass@k 卡住代码正确性，作为 Agent 上线前的回归测试
视觉入口	StructuralEval（范式）	把「图片/图纸→结构 JSON→FrameCalc 求解」封装为 Agent 视觉前端（注意仅杆系 + CC BY-NC）
领域化锚点	调研报告 §1（NASA CRM / wingbox / AGARD 445.6 等）	把教学题映射到真实飞机 FE 模型，桥接 NASTRAN BDF / Abaqus INP

5.2 更新栈（用 FRAME 思路 + §2 试验数据）

阶段	用到的数据	动作
试验/监测输入	§2：Hawk T1A 全机模态、M001 CRM 折叠翼 GVT + 更新后模型、复合材料导波库	模态频率/振型/导波信号作为更新真值
解读与建议	FRAME（思路，待下载）	对分析结果与试验偏差做 Identify/Extract/Recommend，定位需修正参数
参数反演	自研 + NAFEMS/教材基准	以偏差为目标做模型修正（有限元模型更新 / 贝叶斯反演）

5.3 推荐落地路线

1. **第 1 步**：用 FEM-Bench 给你的基座/候选模型建「代码正确性基线」。
2. **第 2 步**：用 ALL-FEM 做 SFT，得到建模代码助手，再回测 FEM-Bench 看提升。
3. **第 3 步**：用 StructuralEval 验证「视觉→模型 JSON」前端，纳入 Agent 工具集（研究用途，遵守 CC BY-NC）。
4. **第 4 步**：以 §1/§2 真实飞机 FE 模型 + 试验数据为锚，做领域化扩充与「构建→更新」闭环原型。
5. **第 5 步**：FRAME 链接一旦取得，把其解读/建议能力接进更新闭环（Identify→Extract→Recommend）。

6. 注意事项与许可红线

可商用 / 自由使用：ALL-FEM（依论文开源声明）、FEM-Bench（见其 LICENSE）。

非商用（务必注意）：StructuralEval 为 **CC BY-NC 4.0**，仅限非商业研究；任何商用产品或对外服务前需联系作者获取授权。

待确认：FRAME 许可未明，且其数据集当前无法获取；使用前须确认许可与可下载性。

环境注意：本工作沙箱 **github.com 不可达**，GitHub 托管的数据集需经镜像（HF→hf-mirror.com，FEM-Bench→Zenodo 存档）下载，内容与原仓库一致。

领域化是硬需求：四个基准均为**通用计算力学 / 非飞机专用**，落地到「飞机结构强度分析」必须做几何/材料/工况的领域化扩充，并桥接工业求解器格式（BDF/INP）。它们解决「构建与解读」，**不解决「更新」**——更新数据请回到调研报告 §2。

—— 本指南与《飞机结构FEA数据集调研报告》（MD/HTML/PDF/Word）配套使用。本地数据集见 `datasets/` 目录，含 `datasets/README.md` 下载清单。